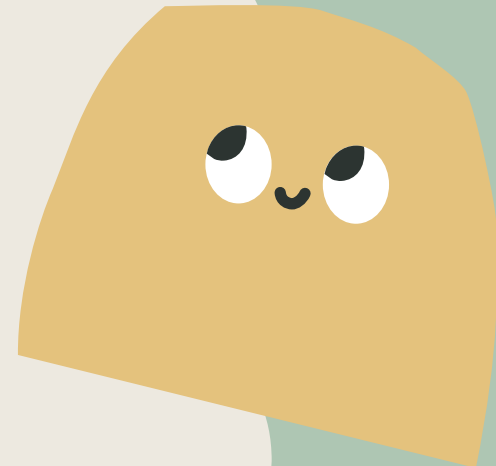
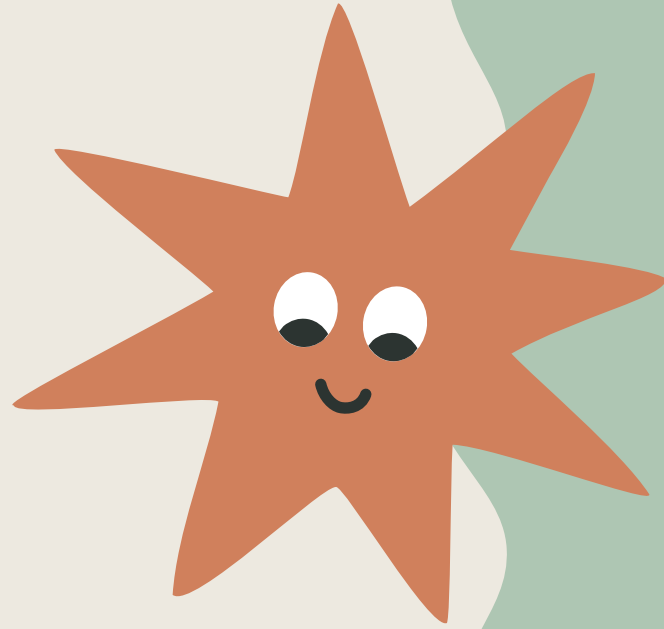


Stack Implementation Using Array in C++

Lecture 13



Learning Outcomes (Lecture 13)

➤ At the end of this lecture, students will be able to:

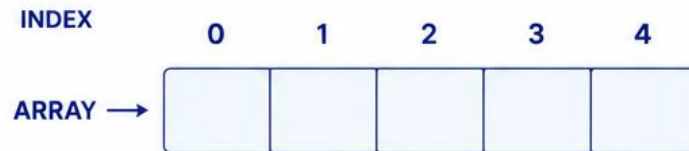
- Implement a Stack using an array
- Create a Stack class
- Understand the purpose of the top variable
- Implement
 - push(), pop(), peek(), isEmpty(), isFull(), display()
- Handle Stack Overflow and Underflow
- Understand the relationship between array indexes and Stack top
- Analyze the time complexity of each operation

3. HOW CAN AN ARRAY REPRESENT A STACK?

We can use an **array of fixed size** to implement a stack.
We keep a variable **TOP** to track the index of the topmost element.

CREATE AN ARRAY

Suppose the stack capacity is 5.
We create an array of **size 5**.



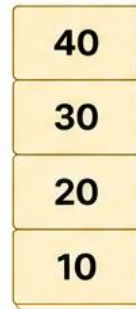
Initially, **TOP = -1** (Stack is empty)



TOP stores the index of the current top element.
-1 means there is no element in the stack.

CONCEPT

TOP



TOP
(Topmost Element)

STACK (LIFO)

MAPPING WITH ARRAY

Stack elements are stored in the array from **index 0** upward.

TOP always points to the last inserted element (the top of the stack).

General Rules:

- When we **PUSH**, **TOP** is increased.
- When we **POP**, **TOP** is decreased.
- Elements exist in the array from index 0 to **TOP**.
- Elements above **TOP** do not belong to the stack.

PUSH OPERATIONS (EXAMPLE)

1 Initial State

TOP = -1



↑
TOP

2 PUSH(10)

TOP = 0



↑
TOP

3 PUSH(20)

TOP = 1



↑
TOP

4 PUSH(30)

TOP = 2



↑
TOP

5 PUSH(40)

TOP = 3



↑
TOP



The stack grows from left to right in the array. **TOP** always points to the index of the topmost element.

Why Does top Start at -1?

- Suppose we started with: `top = 0;`
- Then how would we distinguish
 - Empty stack
 - Stack containing the first element at index 0?

Stack Capacity

➤ Suppose:

```
const int SIZE = 5;
```

➤ Valid array indexes are:

```
0 1 2 3 4
```

➤ Therefore the Stack is full when:

```
top == SIZE - 1
```

isEmpty()

Algorithm:

1. If `top == -1`
 return TRUE
2. Otherwise
 return FALSE
3. Stop

C++:

```
bool isEmpty()  
{  
    return top == -1;  
}
```

Complexity

O(1)

isFull()

Algorithm:

1. If `top == SIZE - 1`
 return `TRUE`
2. Otherwise
 return `FALSE`
3. Stop

C++:

```
bool isFull()
{
    return top == SIZE - 1;
}
```

Complexity

O(1)

push()

Algorithm:

1. Check if Stack is full.
2. If full:
 Display Overflow.
 Stop.
3. Increase TOP.
4. Store value at arr[TOP].
5. Stop.

C++:

```
void push(int value)
{
    if (isFull())
    {
        cout << "Stack Overflow!";
        return;
    }

    top++;
    arr[top] = value;
}
```

Complexity

O(1)

pop()

Algorithm:

1. Check whether Stack is empty.
2. If empty:
 Display Underflow.
 Stop.
3. Store arr[TOP].
4. Decrease TOP.
5. Return removed value.
6. Stop.

C++:

```
int pop()
{
    if (isEmpty())
    {
        cout << "Stack Underflow!";
        return -1;
    }

    int value = arr[top];
    top--;

    return value;
}
```

Complexity

O(1)

peek()

Algorithm:

1. Check if Stack is empty.
2. If empty:
Underflow.
3. Otherwise:
Return arr[TOP].
4. Stop.

C++:

```
int peek()
{
    if (isEmpty())
    {
        cout << "Stack is empty!";
        return -1;
    }

    return arr[top];
}
```

Complexity

O(1)

display()

Algorithm:

1. If Stack is empty:
 Display "Stack is empty".
2. Otherwise:
 Start from TOP.
 Display elements down to index 0.
3. Stop.

C++:

```
void display()
{
    if (isEmpty())
    {
        cout << "Stack is empty!";
        return;
    }

    cout << "Stack:\n";

    for (int i = top; i >= 0; i--)
    {
        cout << arr[i] << endl;
    }
}
```

Complexity

O(n)

One Important C++ Issue

➤ The above implementation uses:

- return -1
- when pop() or peek() fails

➤ This is acceptable for our simple classroom implementation, but in a real application, -1 may itself be a valid data value

➤ For example:

```
Stack contains:
```

```
10
```

```
-1
```

```
20
```

One Important C++ Issue

➤ So returning -1 cannot reliably distinguish:

- **“The actual value is -1”**
- From
- **“An error occurred”**

➤ Professional implementation may use

- Exceptions
- Boolean success flags
- Output/reference parameters
- `std::optional`